

Day 2: Lab Exercises — Thread-Based & Process-Based Parallelism

Duration: 4 hours (self-paced hands-on practice)

Reference: [day2/README.md](#), [day2/README-session1.md](#), and [day2/README-session2.md](#)

Note: These exercises are for personal practice to reinforce the concepts covered today. Start from Exercise 1 and work sequentially.

Exercise 1: Reentrant Locks (RLock) & Nested Resource Access

Topics: `threading.Lock` vs `threading.RLock`, nested locking, avoiding deadlocks

Problem Statement

Write a script `ex01_rlock_recursion.py` demonstrating the difference between `Lock` and `RLock`:

1. Define a class `FolderScanner`:

- Contains a simulated directory hierarchy (represented as nested dictionaries or lists).
- Has a method `scan_directory(folder)` that processes files in the folder and recursively calls itself for sub-folders.
- Maintains a shared integer counter `files_scanned` that must be protected by a lock whenever it is incremented.

2. Test with a standard `threading.Lock`:

- Notice how a thread deadlocks when the recursive call attempts to acquire the lock a second time while already holding it!

3. Replace `threading.Lock` with `threading.RLock`:

- Verify that the thread can now acquire the lock multiple times at each recursion level without hanging.
- Assert that the final count of scanned files matches the expected total.

Hints & Guidelines

- An `RLock` tracks the ID of the thread currently holding it and maintains an internal acquisition count.
- The lock is only released to other threads when the acquisition count returns to zero.
- Always use `with rlock:` context managers to guarantee symmetric release across recursive stack frames.

Best Practices

- Use standard `Lock` by default for mutual exclusion; switch to `RLock` only when methods within the same class call each other while holding a lock.

- Never share an RLock between different threads expecting one thread to release another thread's lock.
-

Exercise 2: Connection Pool Rate-Limiter with Semaphores

Topics: `threading.Semaphore`, resource throttling, concurrent access limiting

Problem Statement

Write a script `ex02_semaphore_pool.py` that simulates a database connection pool:

1. Create a simulated database connection pool:
 - Only **3 concurrent database connections** are allowed at any time.
 - Use `threading.Semaphore(3)` to manage connection slots.
2. Create 10 worker threads representing incoming customer requests.
 - Each worker must acquire a semaphore slot before "connecting".
 - Once connected, print: `"[Worker X] Connected to Database (active slots: Y)"`.
 - Simulate a query taking between 0.3 and 0.8 seconds (`time.sleep()`).
 - Print: `"[Worker X] Finished query, releasing connection"`.
 - Release the semaphore slot.
3. Run the program and verify in the console output that at no point in time do more than 3 workers hold database connections concurrently.

Hints & Guidelines

- Use `with semaphore:` to automatically acquire before the query and release after completion.
- You can inspect the remaining slots using `semaphore._value` for debugging prints.

Best Practices

- Semaphores are ideal for throttling external rate-limited APIs, database connection pools, or GPU access.
 - Avoid holding a semaphore while performing long, unrelated CPU operations.
-

Exercise 3: Phased Multi-Stage Execution with `threading.Barrier`

Topics: `threading.Barrier`, phase synchronization, multi-threaded algorithms

Problem Statement

Write a script `ex03_barrier_phased_compute.py` that coordinates a 4-thread calculation across two distinct phases:

1. Create 4 worker threads and a shared list `data = [0, 0, 0, 0]`.
2. Initialize a barrier: `barrier = threading.Barrier(4)`.
3. **Phase 1 (Local Computation):**

- Each worker i generates a random integer and writes it to `data[i]`.
- Print: `"[Thread i] Phase 1 complete, waiting at barrier..."`.
- Call `barrier.wait()`.

4. Phase 2 (Global Aggregation):

- After all 4 threads pass the barrier, each thread reads the **entire** `data` array and computes its relative percentage of the total sum.
- Print: `"[Thread i] Share of total: X%"`.

Hints & Guidelines

- `barrier.wait()` blocks until exactly 4 threads have arrived, then unblocks all of them simultaneously.
- If one thread crashes or raises an exception, the barrier can be broken using `barrier.abort()`, releasing all waiting threads with a `threading.BrokenBarrierError`.

Best Practices

- Barriers are commonly used in iterative physics simulations, cellular automata, and parallel matrix relaxation algorithms.
- Always handle `BrokenBarrierError` to prevent threads from hanging permanently if a peer fails.

Exercise 4: Multi-Stage Inter-Process Pipeline (Queue & Pipe)

Topics: `multiprocessing.Process`, `multiprocessing.Queue`, `multiprocessing.Pipe`, IPC

Problem Statement

Write a script `ex04_ipc_pipeline.py` implementing a 3-process data transformation pipeline:

```
[Producer Process] —(Queue)—> [Filter Process] —(Pipe)—> [Writer Process]
```

1. Producer Process:

- Generates 15 random integers between 1 and 100.
- Pushes them into a `multiprocessing.Queue`.
- Sends a sentinel value (`None`) when finished.

2. Filter Process:

- Reads integers from the `Queue`.
- Discards odd numbers; keeps only even numbers.
- Sends the filtered even numbers through a `multiprocessing.Pipe` connection (`conn.send(num)`).
- Forwards `None` through the pipe when `None` is encountered.

3. Writer Process:

- Reads filtered numbers from the other end of the **Pipe** (`conn.recv()`).
- Appends them to a local list and prints each accepted value.
- Terminates when **None** is received.

Hints & Guidelines

- Ensure all target functions are defined at the **module level** to comply with macOS **spawn** requirements.
- `multiprocessing.Pipe()` returns two connection handles: (`parent_conn`, `child_conn`). Pass one to the Filter process and the other to the Writer process.
- Guard the process startup in the main block using `if __name__ == '__main__':`.

Best Practices

- Always close unused pipe endpoints in processes to avoid deadlocks when waiting for EOF.
- Use `multiprocessing.Queue` for many-to-one communication, and `Pipe` for high-throughput one-to-one communication.

Exercise 5: Asynchronous Process Pool with Result Callbacks

Topics: `multiprocessing.Pool`, `apply_async()`, asynchronous callbacks, batch processing

Problem Statement

Write a script `ex05_pool_async_callbacks.py` that processes tasks asynchronously without blocking the main process:

1. Create a CPU-bound worker function `process_record(record_id, data_chunk)`:
 - Simulates intensive data processing (e.g., sorting or calculating hashes).
 - Returns a dictionary: `{"record_id": record_id, "processed_elements": len(data_chunk), "checksum": sum(data_chunk)}`.
2. In the main process:
 - Maintain a list `completed_records = []`.
 - Define a callback function `collect_result(result)` that appends the worker's output dictionary to `completed_records` and prints real-time progress.
 - Define an error callback `log_error(err)` that logs any exceptions.
3. Submit 8 separate tasks to a `multiprocessing.Pool()` using `pool.apply_async(process_record, args=(i, chunk), callback=collect_result, error_callback=log_error)`.
4. While the worker pool computes the tasks in the background, have the main process execute a lightweight visual ticker (e.g., printing dots every 0.1 seconds) to prove that the main process is **not blocked**.
5. Close and join the pool, then print the aggregated summary of all processed records.

Hints & Guidelines

- Callbacks in `apply_async` are executed by the **parent process**, making them completely safe for updating parent-process lists or databases.
- Call `pool.close()` followed by `pool.join()` to wait for all asynchronous tasks and their callbacks to finish before exiting.

Best Practices

- Never perform heavy blocking computation inside a callback function—callbacks should be lightweight result collectors.
- Always provide an `error_callback` to ensure unexpected exceptions in child processes don't fail silently.